

Constructive Algorithms

Sean

August 31, 2026

Outline

- The constructive mindset
- CSES Beautiful Permutation
- CSES Gray Code
- Distinct prefix sums modulo n
- Distinct prefix products modulo n
- Patterns, proof, and practice



Introduction

- Constructive problems ask us to explicitly build an object that satisfies a rule.



Introduction

- Constructive problems ask us to explicitly build an object that satisfies a rule.
- The output is not a number or a yes/no answer: it is a certificate that can be checked.

Introduction

- Constructive problems ask us to explicitly build an object that satisfies a rule.
- The output is not a number or a yes/no answer: it is a certificate that can be checked.
- The main skill is finding a simple invariant that makes every output legal.

Introduction

- Constructive problems ask us to explicitly build an object that satisfies a rule.
- The output is not a number or a yes/no answer: it is a certificate that can be checked.
- The main skill is finding a simple invariant that makes every output legal.
- We will build four constructions, from simple grouping to modular design.



1 The Constructive Mindset

2 CSES Beautiful Permutation

3 CSES Gray Code

4 Distinct Prefix Sums Modulo n

5 Distinct Prefix Products Modulo n

6 Patterns, Proof, and Practice



The Constructive Mindset

What Makes a Problem Constructive?

- The statement describes a valid object: a permutation, string, graph, schedule, or sequence.



What Makes a Problem Constructive?

- The statement describes a valid object: a permutation, string, graph, schedule, or sequence.
- Many valid objects may exist; we only need to print one.



What Makes a Problem Constructive?

- The statement describes a valid object: a permutation, string, graph, schedule, or sequence.
- Many valid objects may exist; we only need to print one.
- A checker tests the output, so formatting and bounds are part of correctness.

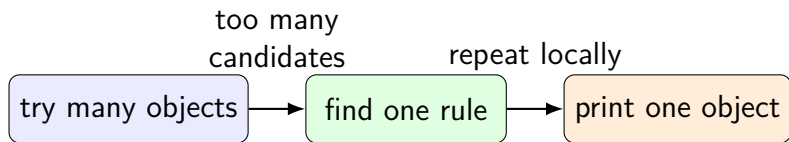


What Makes a Problem Constructive?

- The statement describes a valid object: a permutation, string, graph, schedule, or sequence.
- Many valid objects may exist; we only need to print one.
- A checker tests the output, so formatting and bounds are part of correctness.
- The challenge is to replace search with a rule that generates the object directly.

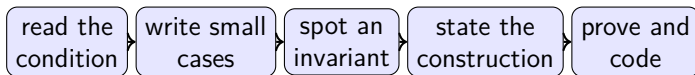


Search Versus Construction



- Brute force is useful for discovering small examples.
- It is usually not the final algorithm when the output space is huge.

A Reliable Construction Workflow



- The proof should explain why the rule works for every input.
- Keep the implementation as close as possible to the sentence describing the rule.

Invariants Are the Safety Rails

- An invariant is a fact that remains true while the construction grows.
- Examples:
 - every printed value is in range and appears once;
 - every new character preserves the required prefix property;
 - every edge keeps the graph connected and acyclic.
- A good construction makes the invariant obvious after each step.

Useful question

What becomes impossible to break after I make this choice?



Small Cases Are a Discovery Tool

Input size	Example / attempt	What to notice
1	1	the base case
2	2, 1 (fails)	the constraint may be impossible local fixes can collide grouping by parity works
3	2, 1, 3 (fails)	
4	2, 4, 1, 3	

- Small cases suggest the shape of a rule.
- They do not replace a proof: the first failure may be at a boundary.



- 1 The Constructive Mindset
- 2 CSES Beautiful Permutation
- 3 CSES Gray Code
- 4 Distinct Prefix Sums Modulo n
- 5 Distinct Prefix Products Modulo n
- 6 Patterns, Proof, and Practice



CSES Beautiful Permutation

Beautiful Permutation

CSES: Permutations

Given n , print a permutation of $1, 2, \dots, n$ in which adjacent values never differ by 1.

- Output any valid permutation, or NO SOLUTION if none exists.
- The output has n values, so an $O(n)$ construction is the natural target.



First Observations

- For $n = 1$, output 1.
- For $n = 2$, both permutations contain the pair 1, 2.
- For $n = 3$, every permutation has two consecutive values next to each other.

Small impossible cases

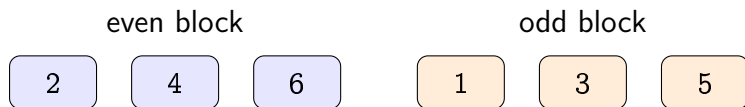
$n = 2$ and $n = 3$ have no solution.

- For $n \geq 4$, we need to keep consecutive values apart.



Group Values by Parity

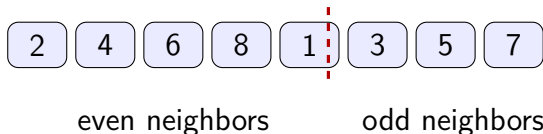
- Two even values differ by at least 2.
- Two odd values differ by at least 2.
- Therefore, put all evens together and all odds together.



The Construction

Rule

Print 2, 4, 6, ... followed by 1, 3, 5,



- Within each block, adjacent values differ by at least 2.
- The only remaining check is the boundary between the blocks.

The Boundary Check

- The last even is at least 4 when $n \geq 4$.
- The first odd is 1.
- Their difference is at least 3, so the blocks also connect safely.

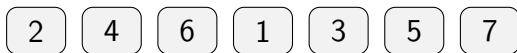
Proof in one sentence

Every adjacent pair is either inside one parity block or crosses a boundary with difference at least 3.

- This is the invariant: the parity blocks keep consecutive values apart.

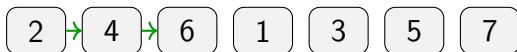


Trace: $n = 7$



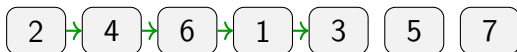
- Print the even block first: 2, 4, 6.

Trace: $n = 7$



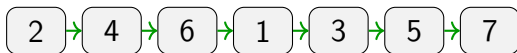
- Print the even block first: 2, 4, 6.
- Inside the even block, every difference is 2.

Trace: $n = 7$



- Print the even block first: 2, 4, 6.
- Inside the even block, every difference is 2.
- The boundary is $6 \rightarrow 1$, with difference 5.

Trace: $n = 7$



- Print the even block first: 2, 4, 6.
- Inside the even block, every difference is 2.
- The boundary is $6 \rightarrow 1$, with difference 5.
- Inside the odd block, every difference is 2.

Beautiful Permutation: Code

```
1  int main() {
2      int n;
3      cin >> n;
4
5      if (n == 2 || n == 3) {
6          cout << "NO SOLUTION\n";
7          return 0;
8      }
9
10     for (int x = 2; x <= n; x += 2) cout << x << ' ';
11     for (int x = 1; x <= n; x += 2) cout << x << ' ';
12     cout << '\n';
13 }
```

- Time: $O(n)$; extra space: $O(1)$.



Construction Checklist

- Feasibility: handle exactly the impossible inputs.
- Coverage: print every required value once.
- Local condition: check neighbors inside each block.
- Boundary condition: check where blocks meet.
- Complexity: generate the output directly.

Takeaway

Grouping by parity turns a global-looking permutation condition into local checks.



- 1 The Constructive Mindset
- 2 CSES Beautiful Permutation
- 3 CSES Gray Code**
- 4 Distinct Prefix Sums Modulo n
- 5 Distinct Prefix Products Modulo n
- 6 Patterns, Proof, and Practice



CSES Gray Code

Gray Code

CSES: Gray Code

Given n , print all 2^n binary strings of length n so consecutive strings differ in exactly one bit.

- Any valid ordering is accepted.
- The output is large, so $O(n2^n)$ time is unavoidable just to print it.

Start with Tiny Gray Codes

n	One valid ordering
1	0, 1
2	00, 01, 11, 10
3	000, 001, 011, 010, 110, 111, 101, 100

- The first half is a smaller Gray code with a leading 0.
- The second half is the reverse smaller code with a leading 1.



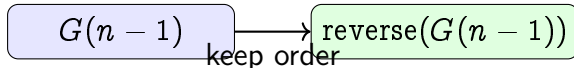
Build $G(n)$ from $G(n - 1)$

Recursive construction

$G(n) = 0 G(n - 1)$ followed by $1 \text{ reverse}(G(n - 1))$.

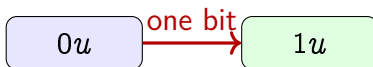
0 + each string

1 + each string



Why Reverse the Second Half?

- Inside the first half, the leading bit is fixed at 0.
- Inside the second half, the leading bit is fixed at 1.
- Reversing preserves the one-bit neighbor property inside the second half.
- At the join, the two suffixes are equal, so only the leading bit changes.

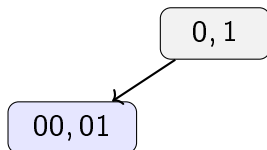


Trace: Build $G(2)$

0, 1

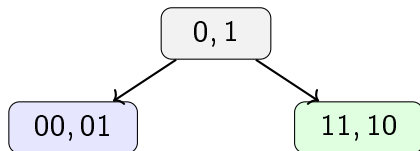
- Start with the one-bit ordering.

Trace: Build $G(2)$



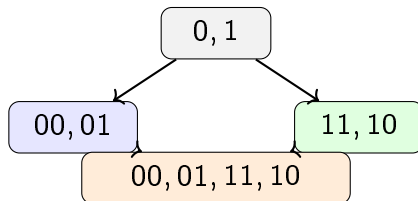
- Start with the one-bit ordering.
- Prefix 0 in the original order.

Trace: Build $G(2)$



- Start with the one-bit ordering.
- Prefix 0 in the original order.
- Prefix 1 in reverse order.

Trace: Build $G(2)$



- Start with the one-bit ordering.
- Prefix 0 in the original order.
- Prefix 1 in reverse order.
- Join the two halves.

Trace: Build $G(3)$

Step	First half	Second half
$G(1)$	0, 1	
$G(2)$	00, 01	11, 10
$G(3)$	000, 001, 011, 010	110, 111, 101, 100

- The second half is the reversed suffix order with a new leading 1.
- Every join changes only the new leading bit.



Gray Code: Code

```
1  vector<string> gray(int n) {
2      if (n == 0) return {" "};
3
4      vector<string> prev = gray(n - 1);
5      vector<string> answer;
6
7      for (string s : prev) answer.push_back("0" + s);
8      for (auto it = prev.rbegin(); it != prev.rend(); ++it)
9          answer.push_back("1" + *it);
10
11     return answer;
12 }
```

- The recursive code mirrors the construction statement.



Gray Code: Correctness

- Base case: $G(0)$ contains the empty string.
- Both halves inherit the one-bit neighbor property from $G(n - 1)$.
- The join is $0u \rightarrow 1u$, so exactly the leading bit changes.
- By induction, each suffix occurs once; prefixing 0 and 1 covers every string exactly once.

Complexity

2^n strings are printed, each of length n : $O(n2^n)$ time and output-sized memory.



Compare the Beginner Constructions

	Parity blocks	Gray code
Goal	keep adjacent values apart	change one bit at a time
Rule	separate by parity	prefix + reverse
Invariant	local differences are safe	neighboring strings differ in one bit
Proof	blocks and boundary	induction and join

- Different-looking problems can share the same workflow: examples, invariant, direct output, proof.



- 1 The Constructive Mindset
- 2 CSES Beautiful Permutation
- 3 CSES Gray Code
- 4 Distinct Prefix Sums Modulo n**
- 5 Distinct Prefix Products Modulo n
- 6 Patterns, Proof, and Practice



Distinct Prefix Sums Modulo n

Distinct Prefix Sums

Problem

Given a permutation $p_1, p_2, \dots, p_n$ of $1, 2, \dots, n$, define

$$s_i = p_1 + p_2 + \dots + p_i.$$

Construct a permutation whose prefix sums are pairwise distinct modulo n , or report that none exists.

- We need a direct rule: the output already contains n values.
- First characterize exactly which values of n are possible.



A Forced First Value

- Suppose $p_i = n$ for some $i > 1$.
- Adding n does not change a residue modulo n :

$$s_i = s_{i-1} + n \equiv s_{i-1} \pmod{n}.$$

- Two consecutive prefix residues would be equal.

Necessary choice

Every valid construction must start with $p_1 = n$.



Odd Sizes Are Impossible

Assume $n > 1$ is odd.

- Since $p_1 = n$, the first residue is $s_1 \equiv 0 \pmod{n}$.
- The final sum is fixed for every permutation:

$$s_n = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

- Because $n+1$ is even, s_n is a multiple of n .
- Thus $s_1 \equiv s_n \equiv 0 \pmod{n}$, a collision.

Feasibility

A solution can exist only when $n = 1$ or n is even.



Discover the Even Pattern

n	Permutation	Prefix residues
2	2, 1	0, 1
4	4, 1, 2, 3	0, 1, 3, 2
6	6, 1, 4, 3, 2, 5	0, 1, 5, 2, 4, 3

- Alternate a large even value with a small odd value.
- The residues alternate between small positive and negative values.



Prefix-Sum Construction

For even n , define

$$p_i = \begin{cases} n + 1 - i, & i \text{ is odd,} \\ i - 1, & i \text{ is even.} \end{cases}$$



Prefix-Sum Construction

For even n , define

$$p_i = \begin{cases} n + 1 - i, & i \text{ is odd,} \\ i - 1, & i \text{ is even.} \end{cases}$$

This prints

$$n, 1, n - 2, 3, n - 4, 5, \dots, 2, n - 1.$$

- Odd positions walk downward through the even values.
- Even positions walk upward through the odd values.



Trace: Prefix Sums for $n = 8$

i	1	2	3	4	5	6	7	8
p_i	8	1	6	3	4	5	2	7
$s_i \bmod 8$	0	1	7	2	6	3	5	4

The hidden shape

$0, 1, -1, 2, -2, 3, -3, 4 \pmod{8}$

- The construction deliberately visits every residue once.



Why the Prefix Sums Are Distinct

The residues have the form

$$0, 1, -1, 2, -2, \dots, \frac{n}{2}.$$

- Positive values range from 1 to $n/2$.
- Negative magnitudes range from 1 to $n/2 - 1$.
- A positive a could equal $-b$ modulo n only if $n \mid a + b$.
- But $1 \leq a + b \leq n - 1$, so no such collision is possible.

Conclusion

A solution exists exactly when $n = 1$ or n is even.



Prefix-Sum Construction: Code

```
1 void solve_prefix_sum(int n) {
2     if (n > 1 && n % 2 == 1) {
3         cout << "No\n";
4         return;
5     }
6
7     cout << "Yes\n";
8     for (int i = 1; i <= n; ++i) {
9         int value = (i % 2 == 1 ? n + 1 - i : i - 1);
10        cout << value << " \n"[i == n];
11    }
12 }
```

- Time: $O(n)$; extra space: $O(1)$.



- 1 The Constructive Mindset
- 2 CSES Beautiful Permutation
- 3 CSES Gray Code
- 4 Distinct Prefix Sums Modulo n
- 5 Distinct Prefix Products Modulo n**
- 6 Patterns, Proof, and Practice



Distinct Prefix Products Modulo n

Distinct Prefix Products

Problem

Given a permutation $p_1, p_2, \dots, p_n$ of $1, 2, \dots, n$, define

$$t_i = p_1 p_2 \cdots p_i.$$

Construct a permutation whose prefix products are pairwise distinct modulo n , or report that none exists.

- The statement looks like the prefix-sum problem.
- Multiplication introduces zero residues and modular inverses.



Both Endpoints Are Forced

- If 1 appears after the first position, multiplying by it repeats the previous prefix product.
- Therefore $p_1 = 1$.
- Once n is multiplied in, every later prefix product is 0 modulo n .
- Therefore $p_n = n$.

Residue target

$t_1, \dots, t_{n-1}$ must be all $n - 1$ nonzero residues; $t_n \equiv 0$.



Why Almost Every Composite Size Fails

Since $p_n = n$, the first $n - 1$ values are $1, 2, \dots, n - 1$ in some order. Hence

$$t_{n-1} \equiv (n - 1)! \pmod{n}.$$

- If $n = ab$ with $2 \leq a < b < n$, both a and b occur in $(n - 1)!$, so $n \mid (n - 1)!$.
- If $n = a^2$ with $a \geq 3$, the distinct factors a and $2a$ occur, so again $n \mid (n - 1)!$.
- The only composite value not covered is $n = 4$.

Consequence

For every composite $n \neq 4$, $t_{n-1} \equiv t_n \equiv 0$: impossible.



Feasible Sizes

Complete characterization

A solution exists exactly when $n = 1$, $n = 4$, or n is prime.

- $n = 1$: use 1.
- $n = 4$: use 1, 3, 2, 4; its prefix residues are 1, 3, 2, 0.
- For prime n , we will force a particularly simple residue order.



Choose the Prefix Products First

For prime n , set the design target

$$t_i \equiv i \pmod{n} \quad (1 \leq i \leq n-1).$$



Choose the Prefix Products First

For prime n , set the design target

$$t_i \equiv i \pmod{n} \quad (1 \leq i \leq n-1).$$

Because $t_i \equiv t_{i-1}p_i \pmod{n}$, we need

$$(i-1)p_i \equiv i \pmod{n}.$$

- Instead of guessing the permutation, prescribe an easy sequence of states.
- Then solve each transition for the next value.



Modular Inverses

An inverse of a modulo n is a value a^{-1} satisfying

$$aa^{-1} \equiv 1 \pmod{n}.$$

- Modulo a prime, every nonzero residue has an inverse.
- Example modulo 7: $2^{-1} \equiv 4$ and $3^{-1} \equiv 5$.
- Fermat's little theorem gives

$$a^{-1} \equiv a^{n-2} \pmod{n}.$$



Prefix-Product Construction

For prime n , use

$$p_1 = 1, \quad p_n = n,$$

$$p_i \equiv i(i-1)^{-1} \pmod{n} \quad (2 \leq i < n).$$

Prefix-Product Construction

For prime n , use

$$p_1 = 1, \quad p_n = n,$$

$$p_i \equiv i(i-1)^{-1} \pmod{n} \quad (2 \leq i < n).$$

Then each transition satisfies

$$t_i \equiv t_{i-1}p_i \equiv (i-1)i(i-1)^{-1} \equiv i \pmod{n}.$$

- The prefix residues become $1, 2, \dots, n-1, 0$.



Trace: Prefix Products for $n = 7$

i	1	2	3	4	5	6	7
p_i	1	2	5	6	3	4	7
$t_i \bmod 7$	1	2	3	4	5	6	0

- Example: $p_3 \equiv 3 \cdot 2^{-1} \equiv 3 \cdot 4 \equiv 5 \pmod{7}$.
- Example: $p_5 \equiv 5 \cdot 4^{-1} \equiv 5 \cdot 2 \equiv 3 \pmod{7}$.



Why Is It a Permutation?

The prefix products are correct, but the printed values must also be distinct.

- Suppose two middle positions have the same value:

$$p_x = p_y = k, \quad 2 \leq x < y \leq n - 1.$$

- From $(i - 1)p_i \equiv i \pmod{n}$,

$$k(x - 1) \equiv x, \quad k(y - 1) \equiv y \pmod{n}.$$

- Subtracting gives

$$(k - 1)(x - y) \equiv 0 \pmod{n}.$$

- Since n is prime and $x - y \not\equiv 0$, this forces $k \equiv 1 \pmod{n}$.



Finish the Permutation Proof

A middle value cannot equal 1:

$$i(i-1)^{-1} \equiv 1 \implies i \equiv i-1 \pmod{n},$$

which is impossible.

- Every middle value is nonzero, so it lies in $1, 2, \dots, n-1$.
- The middle values are distinct and avoid 1.
- There are $n-2$ of them, so they are exactly $2, 3, \dots, n-1$.
- Together with $p_1 = 1$ and $p_n = n$, the output is a permutation.



Number-Theory Helpers

```
bool is_prime(int n) {
    if (n < 2) return false;
    for (int d = 2; 1LL * d * d <= n; ++d)
        if (n % d == 0) return false;
    return true;
}

long long mod_pow(long long a, long long e, long long mod) {
    long long result = 1;
    while (e > 0) {
        if (e & 1) result = result * a % mod;
        a = a * a % mod;
        e >>= 1;
    }
    return result;
}
```



Prefix-Product Construction: Code

```
void solve_prefix_product(int n) {
    if (n == 1) { cout << "Yes\n1\n"; return; }
    if (n == 4) { cout << "Yes\n1 3 2 4\n"; return; }
    if (!is_prime(n)) { cout << "No\n"; return; }

    cout << "Yes\n1 ";
    for (int i = 2; i < n; ++i) {
        long long inverse = mod_pow(i - 1, n - 2, n);
        cout << i * inverse % n << ' ';
    }
    cout << n << '\n';
}
```

- Per test: $O(\sqrt{n} + n \log n)$ time and $O(1)$ extra space.



- 1 The Constructive Mindset
- 2 CSES Beautiful Permutation
- 3 CSES Gray Code
- 4 Distinct Prefix Sums Modulo n
- 5 Distinct Prefix Products Modulo n
- 6 Patterns, Proof, and Practice



Patterns, Proof, and Practice

Reusable Construction Patterns

- **Group by a property:** parity, sign, color, residue, or size.
- **Pair extremes:** a largest item can cancel or protect a smallest item.
- **Use symmetry:** build one half, then mirror or reverse it.
- **Grow safely:** append an item only when the invariant survives.
- **Handle boundaries first:** small inputs and block joins often decide feasibility.



Construction Proof Template

- 1 **Validity:** every required condition holds.
- 2 **Coverage:** all required objects appear, with no duplicates.
- 3 **Boundary:** transitions between blocks or recursive halves are safe.
- 4 **Complexity:** the construction fits the input and output limits.

A useful standard

If the proof cannot be stated clearly, the construction probably is not finished.



Common Mistakes

- Guessing from one example and skipping the general proof.
- Forgetting impossible small cases.
- Checking only inside blocks, not at the joins.
- Printing a value twice or omitting a value.
- Using a construction that is correct but too slow to print.
- Confusing any valid output with the lexicographically smallest output.



How to Test a Construction

- Test every small input, including the first impossible one.
- Check the output mechanically with a separate validator.
- Compare the construction against brute force for tiny sizes.
- Test the largest input for time, output size, and integer bounds.

Debugging habit

First ask which invariant failed; then inspect the code that was supposed to preserve it.



Practice Roadmap

- **Start:** CSES Permutations - parity grouping and boundary checks.
- **Then:** CSES Gray Code - recursive construction and induction.
- **Deepen:** derive the prefix-sum construction from its residue pattern.
- **Challenge:** re-prove the prefix-product construction without reading the formula.
- **Habit:** write a validator and compare against brute force on tiny inputs.
- **Future:** constructive graph designs and interactive constructions.



Summary

- Constructive problems ask for one explicit valid object.
- Use small cases to discover a rule, then name the invariant.
- Separate validity, coverage, boundaries, and complexity in the proof.
- Prefer direct construction whose code mirrors the proof.
- When stuck, try grouping, symmetry, or prescribing easy intermediate states.



The End

Thank you.

© 2026 Sean. Please do not repost or redistribute without permission.

